Deep Learning for Text Data

Embeddings

FRC - EVL

Text as Data

- Text is treated as sequential data
- Deep learning models detect patterns in:
  - Words
  - Sentences
  - Documents
- Similar to computer vision (patterns in pixels)

Traditional Applications

- Document classification
- Sentiment analysis
- Machine translation
- Question answering
- Time series prediction

LLM Applications

- Text generation and summarization
- Conversational AI and assistants
- Semantic understanding and classification
- Search and question answering (RAG)
- Translation and multilingual tasks
- Knowledge extraction (NER, relations)
- Code generation and programming
- Education and tutoring
- Domain-specific applications (health, legal, finance)
- Embeddings, similarity, recommendation
- Multimodal AI
- Agents and automation

From Text to Numbers

- Neural networks require numerical input
- Text must be converted into vectors
- This process is called:

Text Vectorization

Transforming text into numeric tensors

Tokens and Tokenization

- Text is split into units called **tokens**
- Tokenization = breaking text into tokens
- Types of tokens:
  - Words
  - Characters
  - N-grams

Encoding Methods

- Tokens must be mapped to vectors
- Two main approaches:
  - One-hot encoding
  - Embeddings

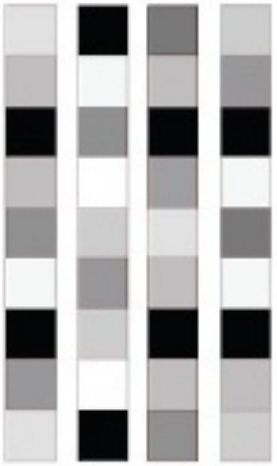
One-Hot Encoding

- Each word = sparse binary vector
- Dimension = vocabulary size
- Limitations:
  - Very high dimensional
  - No semantic meaning
  - No similarity between words

One-hot word vectors:
- Sparse
- High-dimensional
- Hard-coded

Word Embeddings

- Dense, low-dimensional vectors
- Learned from data
- Capture semantic relationships



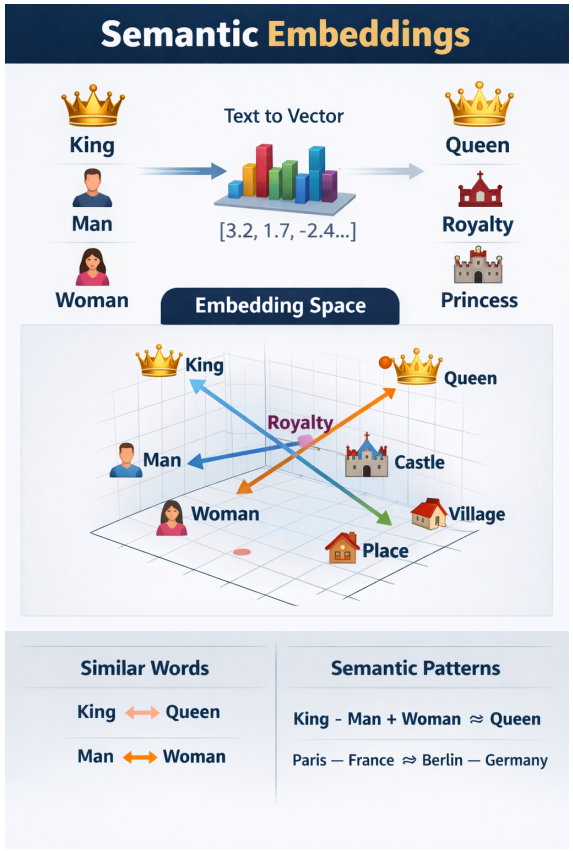
Word embeddings:
- Dense
- Lower-dimensional
- Learned from data

Key Idea

Similar words have similar vectors

Semantic Structure

- Embedding space has geometric meaning
- Distance reflects semantic similarity
- Example relationships:
  - King - Man + Woman $\approx$ Queen
  - Singular $\rightarrow$ Plural transformations



Learning Embeddings

- Embeddings are learned during training
- Start from random vectors
- Updated via backpropagation
- Specialized for the task

Pretrained Embeddings

- Learned from large corpora
- Capture general language structure
- Useful when:
  - Limited training data

Traditional Examples

Word2Vec, GloVe

Embedding Layer

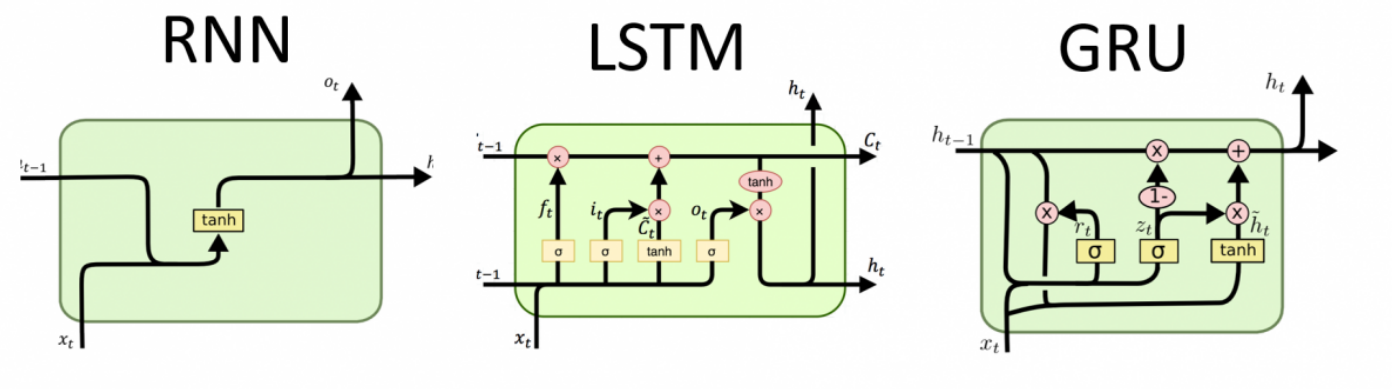
- Maps word indices $\rightarrow$ dense vectors
- Works like a lookup table
- Input:
  - Sequence of integers
- Output:
  - Sequence of vectors

Limitations

- Simple models ignore word order
- No understanding of context
- Same words $\rightarrow$ same vectors in all contexts

Beyond Embeddings

- To capture sequence meaning:
  - Recurrent Neural Networks (RNNs)
  - 1D Convolutional Networks
- These models learn:
  - Word dependencies
  - Sentence structure



Key Takeaways

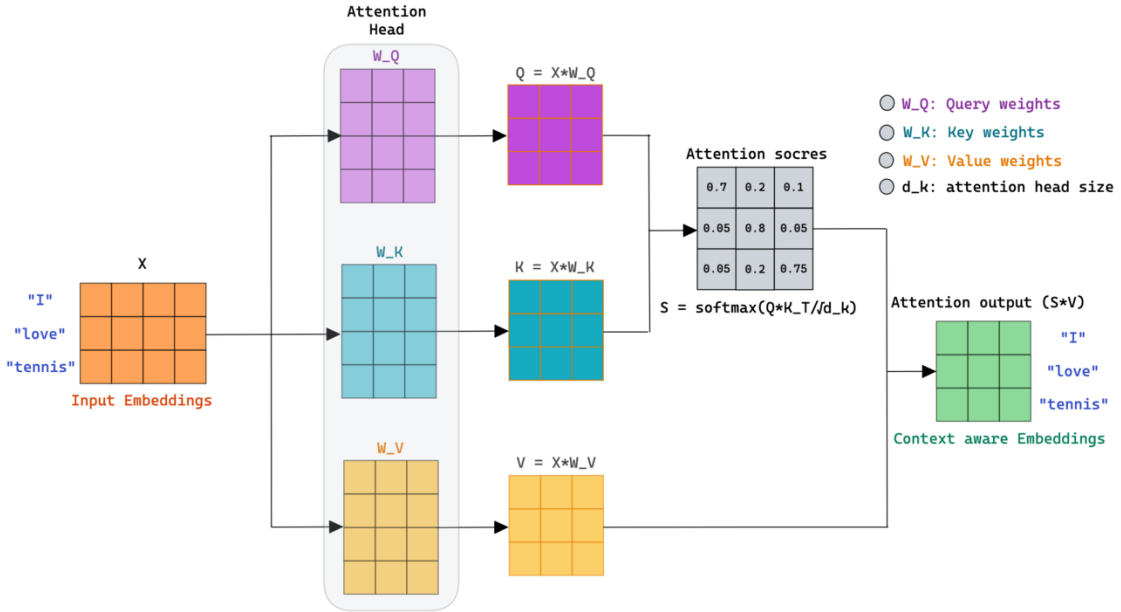
- Text must be vectorized
- Tokenization defines the representation
- Embeddings capture semantics
- Pretrained embeddings improve performance
- Sequence models add context understanding

New Era: From Static to Contextual Embeddings

- Traditional embeddings (Word2Vec, GloVe):
  - One vector per word
  - Same representation in all contexts
- Limitation:
  - "bank" (river vs finance) → same vector
- Solution:

Attention Is All You Need

Contextual Embeddings. Representation depends on the full sentence



Encoder-Only LLMs

- Examples:
  - BERT
  - RoBERTa
  - Sentence-BERT
- Architecture:
  - Transformer encoder
  - Bidirectional context
- Key idea:

Representation Learning

Each token embedding depends on all other tokens

Sentence Embeddings with LLMs

- Encoder models can produce:
  - Token embeddings
  - Sentence embeddings
- Common strategies:
  - Mean pooling over tokens
- Applications:
  - Semantic search
  - Text similarity
  - Clustering
  - Retrieval systems

Why Encoder LLMs Matter

- Capture deep semantic meaning
- Handle polysemy and ambiguity
- Produce transferable representations
- Compared to static embeddings:
  - Context-aware
  - More expressive
  - Better performance in NLP tasks

Modern NLP Pipeline

Text $\rightarrow$ Encoder LLM $\rightarrow$ Embeddings $\rightarrow$ Downstream task